Language Implementation within the Prompt Engineering

Pavlo Bachysh

The Problem: Hard to manage prompts in LLM APIs

- When adding LLMs to applications, developers cannot send just raw text, they must follow strict Chat Completion API schemas.
- This means wrapping every message into complex JSON objects with specific keys (messages, role, content).
- Writing long, multi-line prompts inside rigid JSON files leads to syntax mistakes.

The Solution: My prompt DSL (Making prompt setup easy)

Simpler than raw JSON: No need to manually build heavy, multi-line JSON structures just to send a prompt to an API.

Flexible and quick syntax: You can write messages formally with {} or use super-fast shortcuts like (user): without breaking JSON formatting.

Automatic background check: The compiler automatically turns your clean text file into a valid JSON for OpenAI/Anthropic and catches formatting errors on the fly.

The tool: textX

- The language is built with **textX**, a Python tool for making DSLs.
- It reads the configuration files using a simple grammar file and turns the text into Python objects.

The main idea: From Complex JSON to Simple Strings

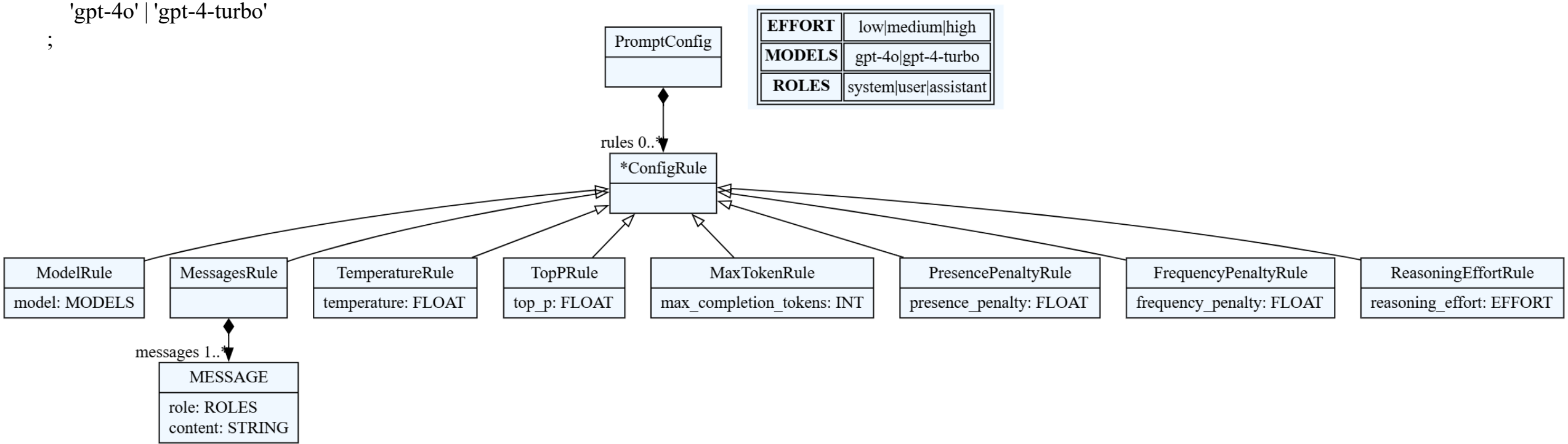
•**The standart way:** Developers have to put every prompt and message into messy, nested JSON structures.

•**My solution:** Users don't write JSON at all. They just type parameters and simple dialogue lines.

grammar.tx

```
PromptConfig:
   rules*=ConfigRule
;

ConfigRule:
   ModelRule | (TemperatureRule | TopPRule) | MaxTokenRule | PresencePenaltyRule |
FrequencyPenaltyRule | ReasoningEffortRule | MessagesRule
;

ModelRule:
   'model' '=' model=MODELS
;

MODELS:
   'gpt-4o' | 'gpt-4-turbo'
;

MessagesRule:
   'messages' '{' messages+=MESSAGE[','] '}'
;

MESSAGE:
   '{'
   'role' ':' role=ROLES ','
   'content' ':' content=STRING
   '}'
   |
   '('role=ROLES')' (':' | '-')? content=STRING
;

ROLES:
   'system' | 'user' | 'assistant'
;

TemperatureRule:
   'temperature' '=' temperature=FLOAT
;

TopPRule:
   'top_p' '=' top_p=FLOAT
;

MaxTokenRule:
   'max_completion_tokens' '=' max_completion_tokens=INT
;

PresencePenaltyRule:
   'presence_penalty' '=' presence_penalty=FLOAT
;

FrequencyPenaltyRule:
   'frequency_penalty' '=' frequency_penalty=FLOAT
;

ReasoningEffortRule:
   'reasoning_effort' '=' reasoning_effort=EFFORT
;

EFFORT:
   'low' | 'medium' | 'high'
;
```

```
PromptConfig:
    rules*=ConfigRule
;

ConfigRule:
    ModelRule | TemperatureRule | TopPRule | MaxTokenRule |
PresencePenaltyRule | FrequencyPenaltyRule | ReasoningEffortRule |
MessagesRule
;

ModelRule:
    'model' '=' model=MODELS
;

MODELS:
    'gpt-4o' | 'gpt-4-turbo'
;

MessagesRule:
    'messages' '{' messages+=MESSAGE[','] '}'
;

MESSAGE:
    '{'
    'role' ':' role=ROLES ','
    'content' ':' content=STRING
    '}'
    |
    '('role=ROLES')' (':' | '-')? content=STRING
;
```



```python
def Limitation(value, min_val, max_val = None):
    if max_val is not None:
        if value < min_val or value > max_val:
            raise ValueError(f"Error, your value {value} but it must be higher than
{min_val} and lower that {max_val}")
        else:
            return value
    else:
        if value <= min_val:
            raise ValueError(f"Error, your value {value} but it must be higher than
{min_val}")
        else:
            return value
```

temperature [0.0; 2.0]
top_p [0.0; 1.0]
max_completion_tokens (0; +inf)
presence_penalty [-2.0; 2.0]
frequency_penalty [-2.0; 2.0]

```
resault = {}
for rule in model.rules:
    rule_type = rule.__class__.__name__
    match rule_type:
        case "ModelRule":
            resault["model"] = rule.model
```

Comparing type of rule for each described rule in prompt with all possible rules and adding it's property to a dictionary

Case handling for the MessagesRule

```
case "MessagesRule":
                messages = rule.messages
                messages_array = []
                for message in messages:
                    mes = {}
                    mes["role"] = message.role
                    mes["content"] = message.content
                    messages_array.append(mes)
                resault["messages"] = messages_array
```

```
 if "top_p" in resault and "temperature" in resault:
        print("It is not recomended to use both top_p and temperature. It may cause some
troubles.")
```

Warning if somebody use both top_p and temperature

```
output_filename = 'output.json'


    with open(output_filename, 'w', encoding='utf-8') as json_file:
        json.dump(resault, json_file, indent=4, ensure_ascii=False)
```

JSON generating

prompt.conf

My prompt

```
model = gpt-4o
temperature = 0.6
max_completion_tokens = 300
frequency_penalty = 0.5
reasoning_effort = low
messages {
    {
        role:system,
        content : "You are a child"
    },
    (user): "What is your favourite animal?",
    (assistant) - "My favourite animal is cat",
    (user)"And why?"
}
```

```
{
    "model": "gpt-4o",
    "temperature": 0.6,
    "max_completion_tokens": 300,
    "frequency_penalty": 0.5,
    "reasoning_effort": "low",
    "messages": [
        {
            "role": "system",
            "content": "You are a child"
        },
        {
            "role": "user",
            "content": "What is your favourite animal?"
        },
        {
            "role": "assistant",
            "content": "My favourite animal is cat"
        },
        {
            "role": "user",
            "content": "And why?"
        }
    ]
}
```

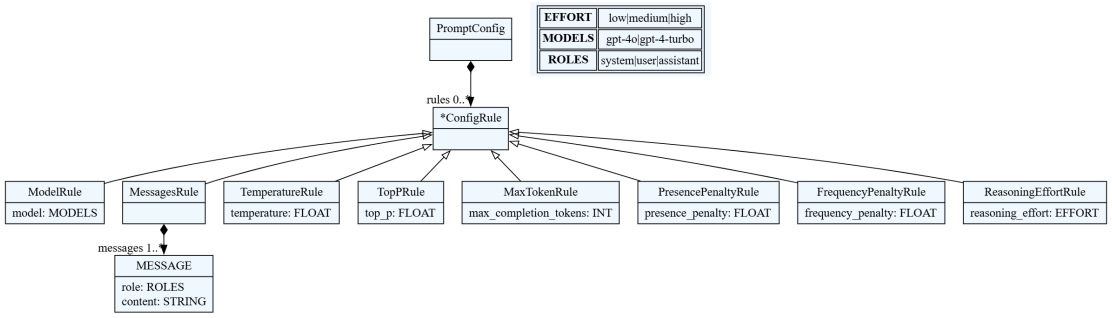

output.json

Output JSON file which was created on my prompt (prompt.conf)

Experiment Evaluation:

•The project successfully proves how SLE concepts can be applied to solve real-world engineering problems in the Generative AI domain.

•Using a DSL drastically improves developer experience compared to writing raw, low-level data formats like JSON.



Key Takeaways from the SLE Approach:

•**Runtime validation:** Moving error detection from runtime to parse-time means parameter bounds and constraints are checked before any execution or network requests.

•**Separation of concrete and abstract syntax:** Thanks to textX, we can offer multiple user-friendly visual styles while parsing them into the exact same unified Abstract Syntax Tree.

•**Model-Driven Generation:** The language compiler serves as a true translator, automating the transformation from a high-level model to a production-ready API payload.